

LoveCode

Memoria Técnica del Proyecto

Alumno: Óscar Reguera Lozano

Ciclo Formativo: 1º DAW

Módulos: Sistemas Informáticos, Bases de Datos, Lenguaje de Marcas, Entorno de Desarrollo, Programación

Profesores:

José Luis López de Pablo Moya
Esteban Castañer Pardo
Pedro Campos Cuevas

Fecha de entrega: 31 de mayo de 2026

Contenido

1. Introducción.....	3
2. Descripción General del Sistema	4
Base de Datos (MariaDB).....	4
Backend (Java / Spring Boot)	4
Frontend (HTML, CSS y JavaScript)	4
Comunicación entre capas	5
3. Diseño de la Base de Datos	6
Tablas principales.....	6
Relaciones entre tablas	7
Trigger de generación de matches	¡Error! Marcador no definido.
4. Desarrollo del Backend	9
Conexión con la base de datos	9
Operaciones del servidor.....	9
Registro y login	10
Gestión de likes y matches.....	13
Generación del XML	14
5. Desarrollo del Frontend.....	18
Estructura general	18
Formulario de registro.....	18
Formulario de login	18
Vista de perfiles	19
Sistema de likes y matches	19
6. Automatización del Sistema	21
Script de backup	21
Script de arranque	¡Error! Marcador no definido.
Automatización con cron	¡Error! Marcador no definido.
7. Funcionamiento del Proyecto	22
Registro de usuario.....	22
Inicio de sesión	22
Visualización de perfiles	22
Dar like y generar un match.....	22
Ficha de compatibilidad	22
8. Problemas Encontrados y Soluciones.....	24
9. Conclusiones.....	26

1. Introducción

LoveCode es una aplicación web inspirada en las plataformas de tipo match, pero orientada al mundo tecnológico. El objetivo es conectar a desarrolladores y profesionales del sector según las tecnologías que conocen o que les interesan.

Los usuarios pueden registrarse indicando las tecnologías que dominan, explorar otros perfiles y expresar interés mediante un sistema de likes. Cuando dos usuarios se dan like mutuamente, el sistema genera automáticamente un match entre ellos, mostrando una ficha de compatibilidad con las tecnologías que tienen en común.

El proyecto integra varias asignaturas del ciclo formativo, permitiendo comprender cómo se relacionan entre sí la base de datos, el backend de la aplicación, la interfaz web y la gestión del entorno de desarrollo.

Las funcionalidades principales de la aplicación son:

- Registro de usuarios con selección de tecnologías
- Inicio de sesión con validación de credenciales
- Visualización de perfiles en formato de tarjetas
- Sistema de likes entre usuarios
- Generación automática de matches cuando el like es mutuo
- Ficha de compatibilidad en formato XML/XSLT

2. Descripción General del Sistema

La aplicación está organizada en tres partes principales que se comunican entre sí:

Base de Datos (MariaDB)

```
oscarrrl@debian:~$ sudo systemctl status mariadb
[sudo] contraseña para oscarrrl:
● mariadb.service - MariaDB 11.8.6 database server
   Loaded: loaded (/usr/lib/systemd/system/mariadb.service; enabled; pre
   Active: active (running) since Sat 2026-05-30 18:03:53 CEST; 23h ago
   Invocation: cf21b11abd114368bac4d30f3ee2a8a0
   Docs: man:mariadb(8)
         https://mariadb.com/kb/en/library/systemd/
   Process: 843 ExecStartPre=/bin/sh -c [ ! -e /usr/bin/galera_recovery ]
   Process: 1019 ExecStartPost=/bin/rm -f /run/mysql/wsrep-start-positio
   Process: 1021 ExecStartPost=/etc/mysql/debian-start (code=exited, stat
   Main PID: 911 (mariabdd)
   Status: "Taking your SQL requests now..."
   Tasks: 19 (limit: 15165)
   Memory: 173M (peak: 173.4M)
   CPU: 42.885s
   CGroup: /system.slice/mariadb.service
           └─911 /usr/sbin/mariabdd
```

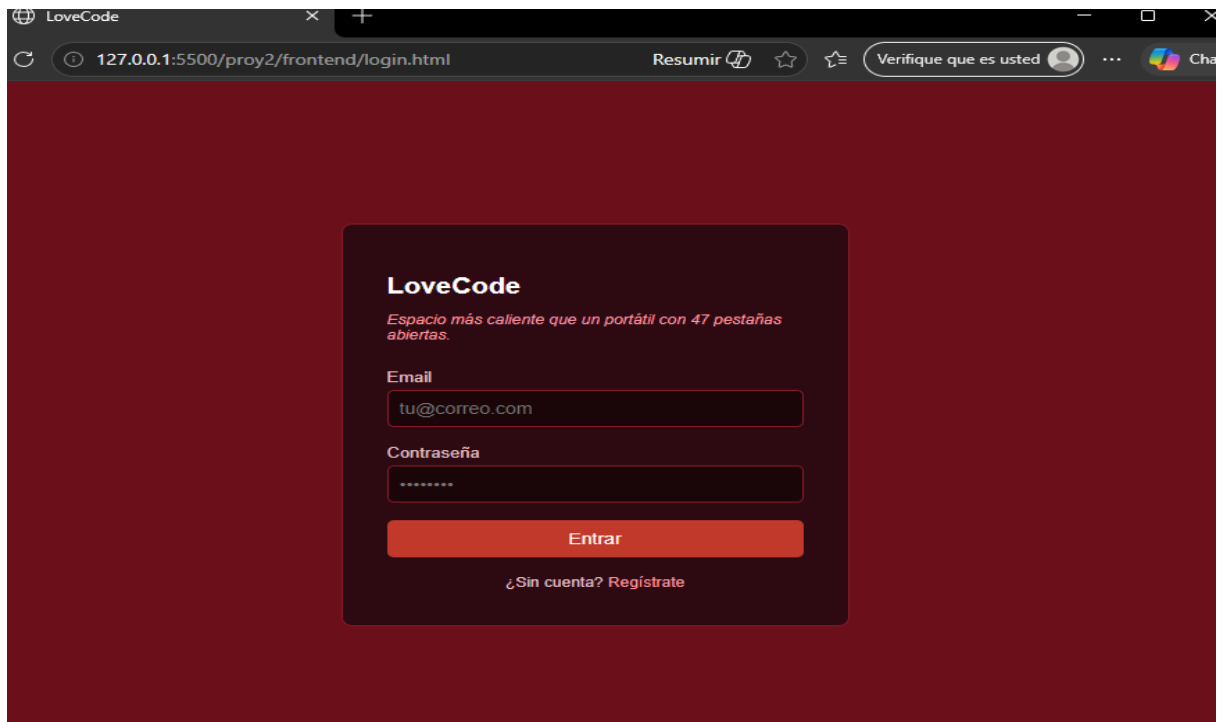
MariaDB 11.8.6 activo en la VM Debian

Se encarga de almacenar toda la información del sistema: usuarios, tecnologías, interacciones entre perfiles, likes y matches. La base de datos relacional organiza la información en tablas relacionadas mediante claves primarias y foráneas. Se ejecuta dentro de una máquina virtual Debian con MariaDB 11.8.6.

Backend (Java / Spring Boot)

Una aplicación desarrollada en Java con Spring Boot actúa como servidor intermedio entre la base de datos y el navegador. Se conecta a la base de datos mediante JDBC y expone una serie de endpoints REST que el frontend puede consumir. Responde en formato JSON y genera archivos XML cuando se produce un match.

Frontend (HTML, CSS y JavaScript)



Interfaz web de LoveCode – página de login

La interfaz web permite a los usuarios interactuar con la aplicación: registrarse, iniciar sesión, visualizar perfiles y dar likes. El navegador realiza peticiones HTTP al servidor para obtener o enviar información, y procesa las respuestas para actualizar la interfaz dinámicamente.

Comunicación entre capas

El frontend se comunica con el backend mediante peticiones fetch en JavaScript. El backend procesa las peticiones, consulta la base de datos y devuelve los resultados en JSON. Cuando se produce un match, el backend genera un XML que el frontend transforma mediante XSLT para mostrar la ficha de compatibilidad.

3. Diseño de la Base de Datos

La base de datos LoveCode está formada por cinco tablas principales que almacenan la información necesaria para el funcionamiento del sistema.

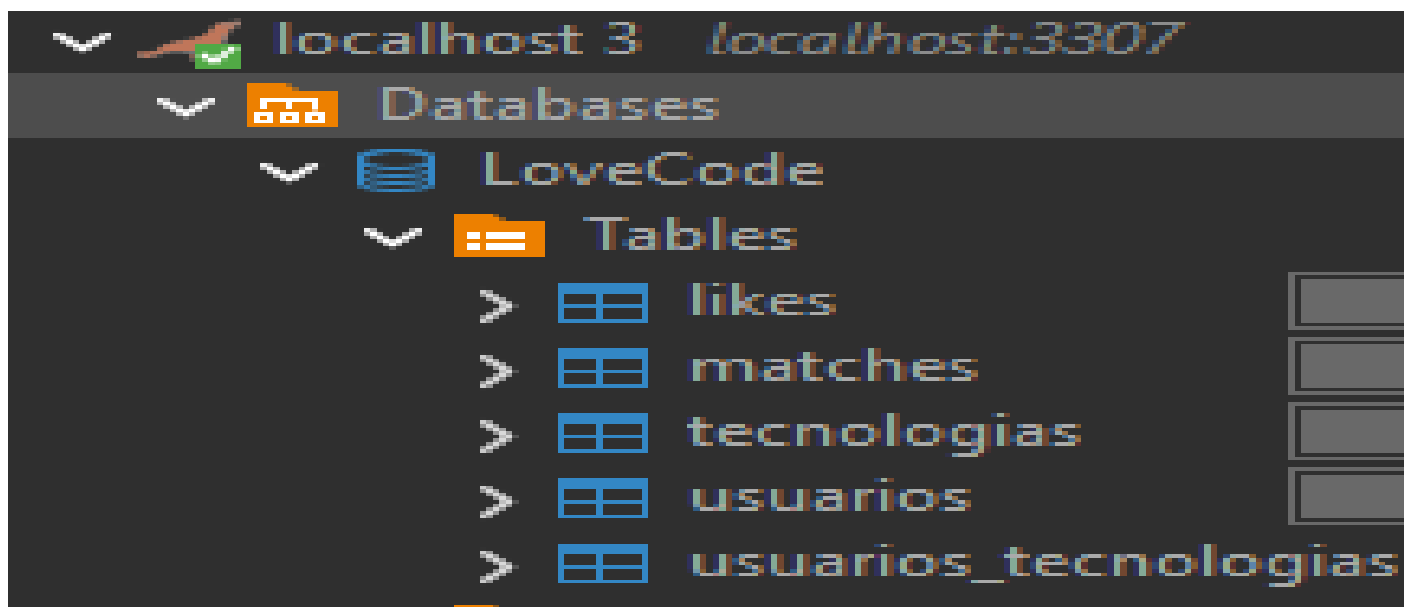
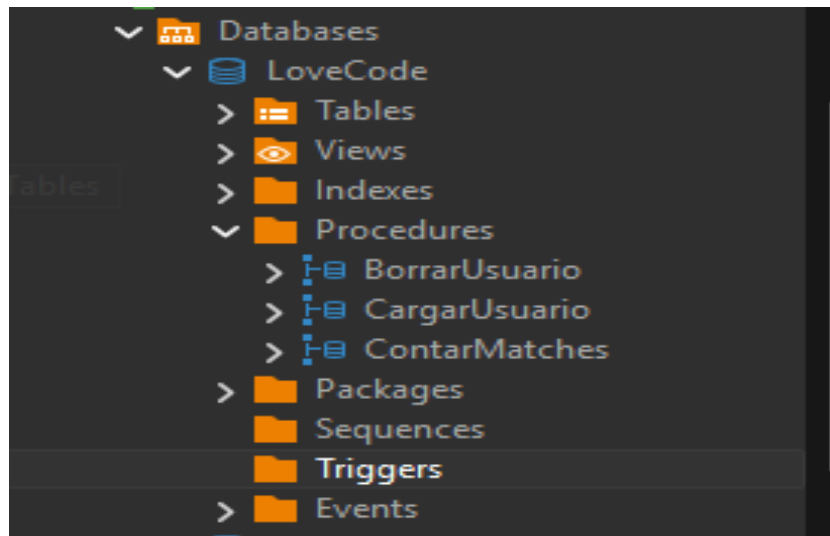
Tablas principales

```
oscarri@debian:~$ mysql -u db_admin -p1234 LoveCode -e "DESCRIBE usuarios;"
```

Field	Type	Null	Key	Default	Extra
id_usuario	int(11)	NO	PRI	NULL	auto_increment
nombre	varchar(100)	NO		NULL	
email	varchar(150)	NO	UNI	NULL	
password	varchar(255)	NO		NULL	
descripcion	text	YES		NULL	
fecha_registro	datetime	YES		current_timestamp()	

```
oscarri@debian:~$
```

la estructura de la tabla **usuarios** de la base de datos: tiene 6 campos (id, nombre, email, contraseña, descripción y fecha de registro).



Estructura de las tablas likes, matches y usuarios_tecnologias

- usuarios: almacena los datos de cada usuario registrado (id, nombre, email, password, descripcion, fecha_registro)
- tecnologias: catálogo de tecnologías disponibles (id, nombre)
- usuarios_tecnologias: tabla intermedia que relaciona usuarios con tecnologías (id_usuario, id_tecnologia)
- likes: registra los likes entre usuarios (id_like, id_emisor, id_receptor, fecha_like)
- matches: almacena los matches generados automáticamente (id_match, id_usuario1, id_usuario2, fecha_match)

Relaciones entre tablas

The screenshot shows a database client window with a tab labeled "<localhost 3> Script-7". The SQL editor contains the query: `SELECT id, nombre, email FROM usuarios;`. Below the editor, a toolbar shows the query has been executed. The results are displayed in a table view with columns: id, nombre, and email. The table contains 7 rows of data.

	id	nombre	email
1	1	Sergio Ruiz Obama	sergio.ruiz.obama@gmail.com
2	3	Kylian Mbappe	mbappe@email.com
3	4	Erling Haaland	haaland@email.com
4	5	Vinicius Junior	vinicius@email.com
5	6	Pedri Gonzalez	pedri@email.com
6	7	Sergio	sergio@gmailuknow.com

*SELECT * FROM usuarios – datos de prueba en la BD*

La tabla usuarios_tecnologias implementa una relación muchos a muchos entre usuarios y tecnologías, ya que un usuario puede conocer varias tecnologías y una tecnología puede ser conocida por varios usuarios. La tabla likes relaciona dos usuarios mediante claves foráneas que apuntan a la tabla usuarios.

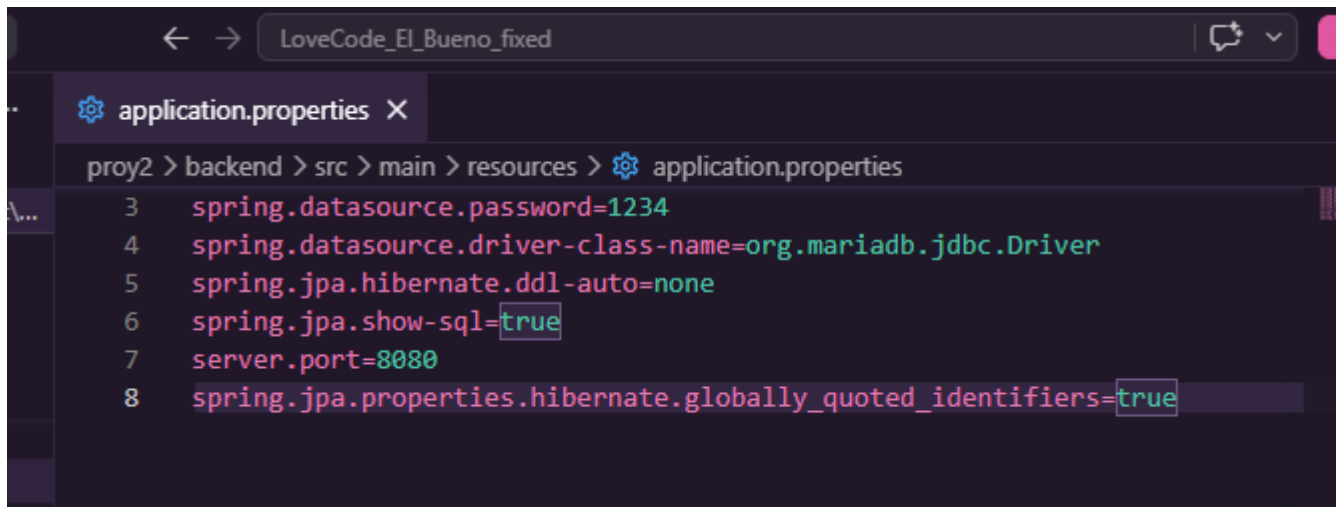
El sistema incluye un trigger llamado tr_generacion_matches que se ejecuta automáticamente después de cada INSERT en la tabla likes. El trigger comprueba si existe un like recíproco: si el usuario receptor también le ha dado like al emisor, inserta automáticamente un registro en la tabla matches con los ids de ambos usuarios y la fecha del match. Además, usa INSERT IGNORE para evitar matches duplicados.

4. Desarrollo del Backend

Conexión con la base de datos

El backend se conecta a la base de datos MariaDB mediante JDBC, configurado en el archivo `application.properties`. La URL de conexión apunta a `localhost:3307`, que mediante port forwarding en VirtualBox redirige al puerto 3306 de la máquina virtual donde corre MariaDB. Se utiliza el pool de conexiones HikariCP, que gestiona automáticamente el ciclo de vida de las conexiones.

Operaciones del servidor

A screenshot of an IDE window titled "LoveCode_El_Bueno_fixed". The editor shows the file "application.properties" with the following content:

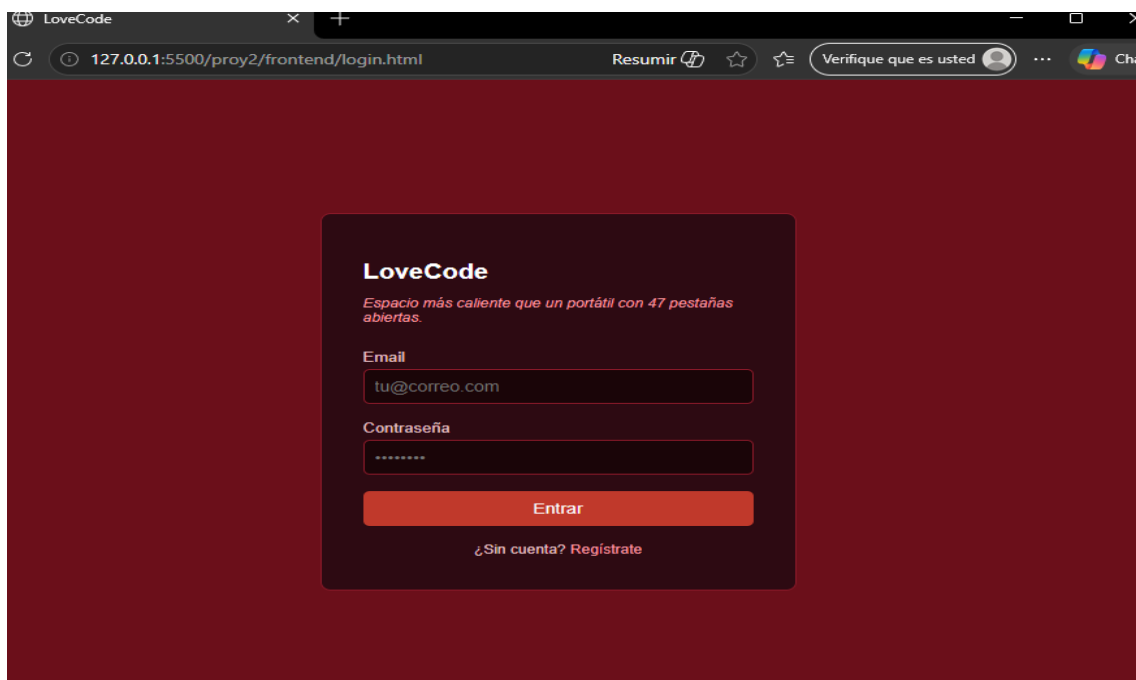
```
3 spring.datasource.password=1234
4 spring.datasource.driver-class-name=org.mariadb.jdbc.Driver
5 spring.jpa.hibernate.ddl-auto=none
6 spring.jpa.show-sql=true
7 server.port=8080
8 spring.jpa.properties.hibernate.globally_quoted_identifiers=true
```

Configuración JDBC en application.properties

El servidor implementa los siguientes endpoints REST en la clase `UsuarioController`:

- POST `/registro`: registra un nuevo usuario y sus tecnologías asociadas
- POST `/login`: valida las credenciales y devuelve el id del usuario
- GET `/perfiles`: devuelve la lista de perfiles excluyendo al usuario en sesión
- GET `/tecnologias`: devuelve el catálogo de tecnologías disponibles
- POST `/like`: registra un like y comprueba si se ha generado un match
- GET `/match-xml`: genera el XML de compatibilidad para un match

Registro y login



LoveCode

Espacio más caliente que un portátil con 47 pestañas abiertas.

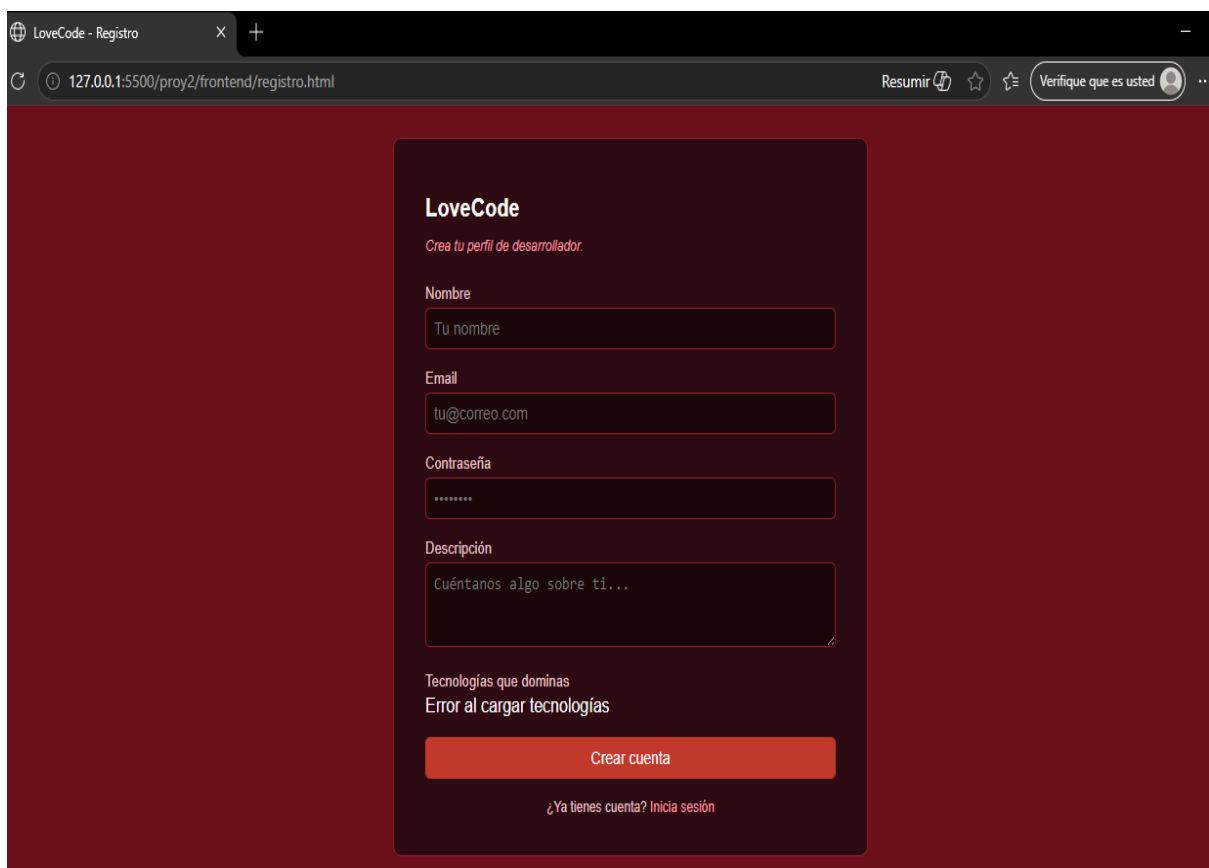
Email
tu@correo.com

Contraseña

Entrar

[¿Sin cuenta? Regístrate](#)

Iniciar sesión



LoveCode

Crea tu perfil de desarrollador.

Nombre
Tu nombre

Email
tu@correo.com

Contraseña

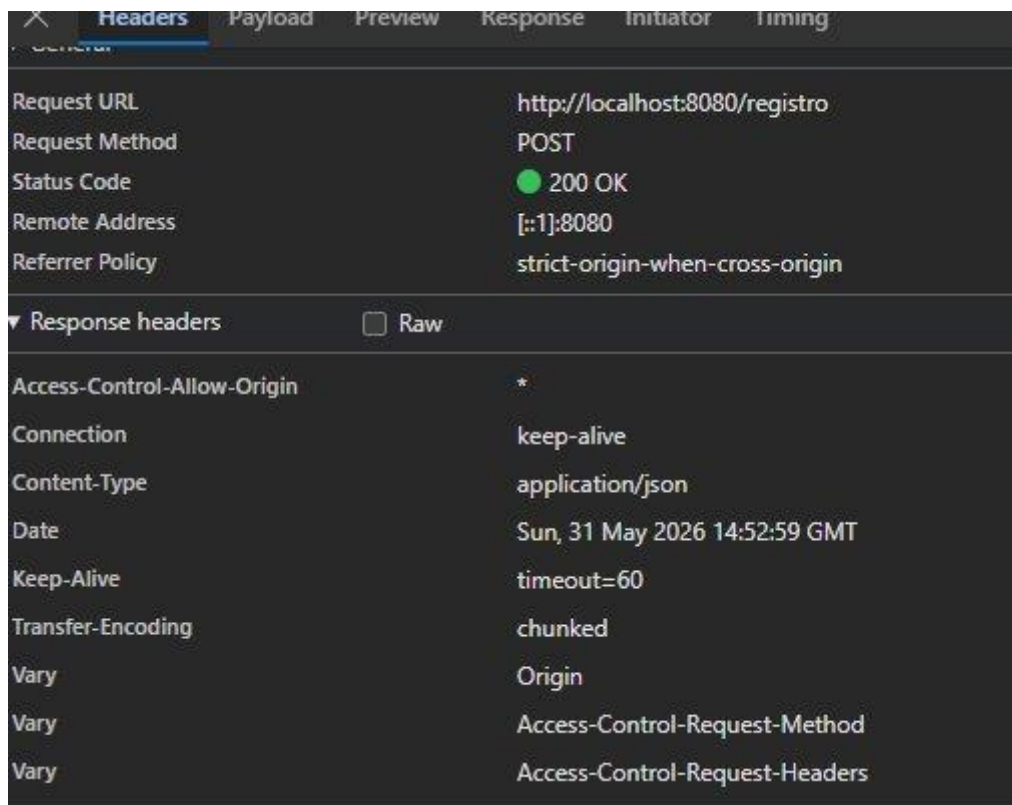
Descripción
Cuéntanos algo sobre ti...

Tecnologías que dominas
Error al cargar tecnologías

Crear cuenta

[¿Ya tienes cuenta? Inicia sesión](#)

Registro en la Web



DevTools Network: POST /registro con respuesta 200 OK
Formulario de registro con tecnologías cargadas

```

Login OK: haaland@email.com
Hibernate: select u1_0.`id`,u1_0.`descripcion`,u1_0.`email`,u1_0.`fecha_registro`,u1_0.`nombre`,u1_0.`password` from `usuarios`
u1_0 where u1_0.`id`<>?
Hibernate: select u1_0.`id`,u1_0.`descripcion`,u1_0.`email`,u1_0.`fecha_registro`,u1_0.`nombre`,u1_0.`password` from `usuarios`
u1_0 where u1_0.`email`=? and u1_0.`password`=?
Hibernate: select u1_0.`id`,u1_0.`descripcion`,u1_0.`email`,u1_0.`fecha_registro`,u1_0.`nombre`,u1_0.`password` from `usuarios`
u1_0 where u1_0.`email`=? and u1_0.`password`=?
Login OK: sergio.ruiz.obama@gmail.com
Hibernate: select u1_0.`id`,u1_0.`descripcion`,u1_0.`email`,u1_0.`fecha_registro`,u1_0.`nombre`,u1_0.`password` from `usuarios`
u1_0 where u1_0.`id`<>?
Match entre 1 y 4

```

The screenshot shows the DevTools Network tab with the 'Headers' sub-tab selected. The left sidebar lists several resources, with 'login' highlighted. The main panel displays the request details for the selected resource.

General	
Request URL	http://localhost:8080/login
Request Method	POST
Status Code	200 OK
Remote Address	[::1]:8080
Referrer Policy	strict-origin-when-cross-origin

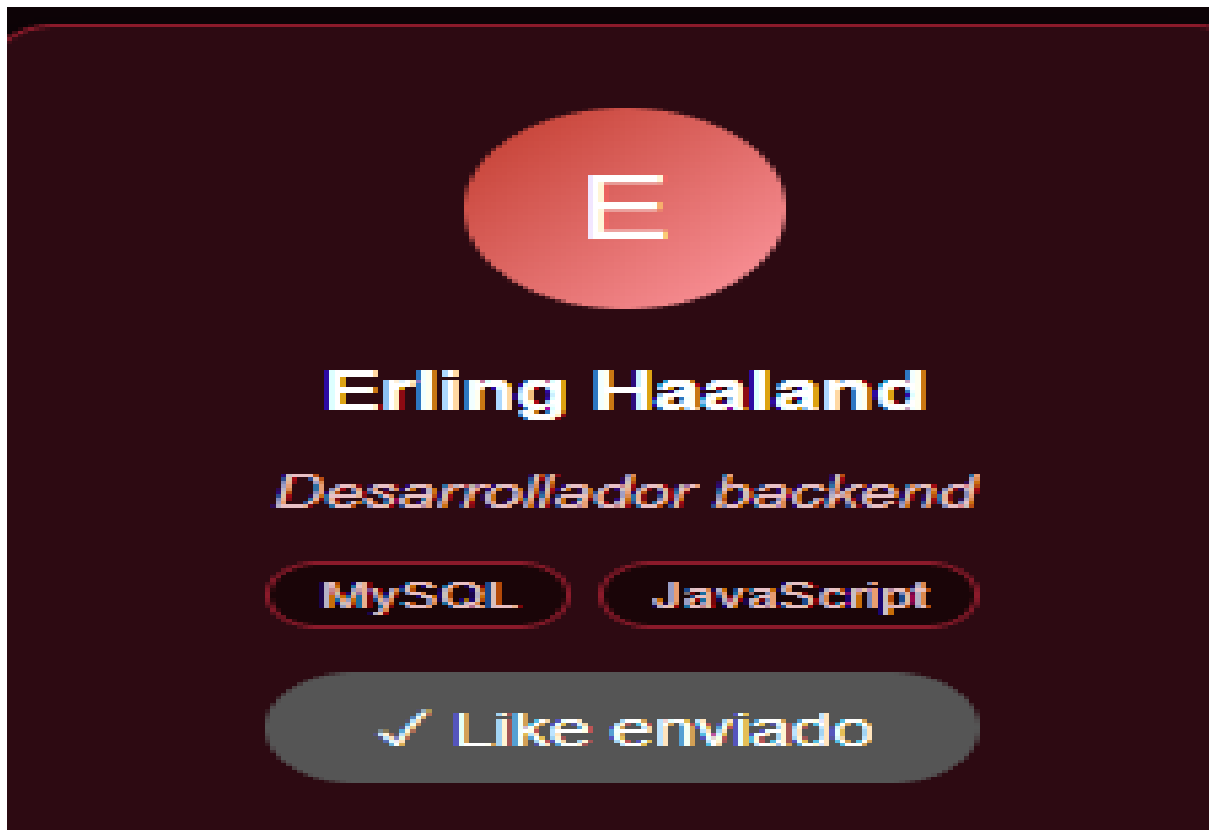
Response headers	
Access-Control-Allow-Origin	*
Connection	keep-alive
Content-Type	application/json
Date	Sun, 31 May 2026 15:04:32 GMT
Keep-Alive	timeout=60
Transfer-Encoding	chunked
Vary	Origin
Vary	Access-Control-Request-Method
Vary	Access-Control-Request-Headers

At the bottom of the network panel, it shows: 2 requests, 2.7 kB transferred, 16...

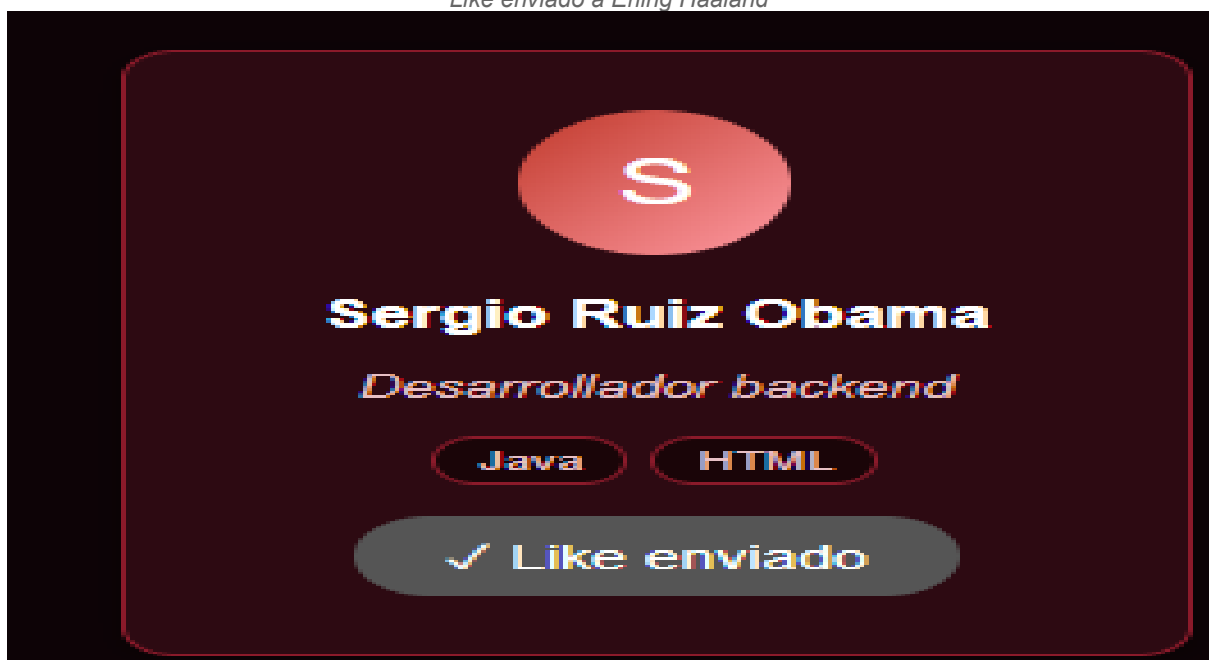
*DevTools Network: POST /login con respuesta 200 OK
Formulario de login con credenciales rellenas*

El endpoint de registro recibe los datos del usuario en formato JSON, comprueba que el email no esté ya registrado, guarda el usuario en la base de datos mediante JPA y luego inserta sus tecnologías en la tabla usuarios_tecnologías. El endpoint de login busca el usuario por email y contraseña, y devuelve su id si las credenciales son correctas.

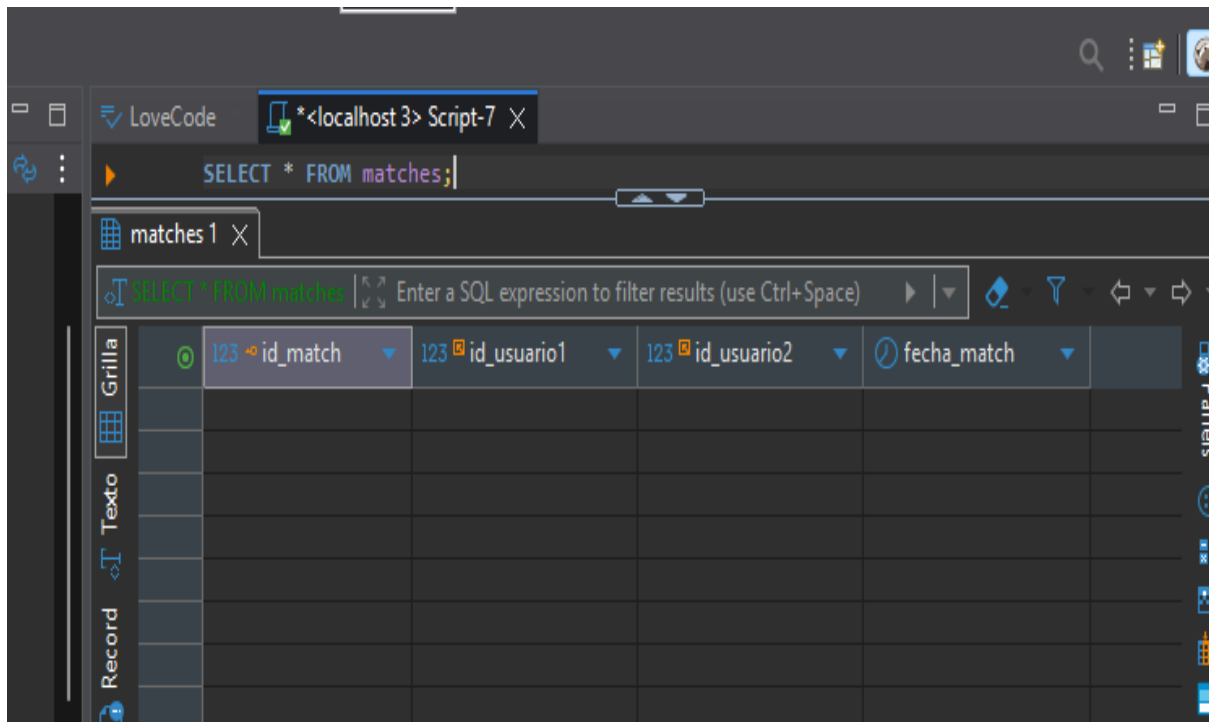
Gestión de likes y matches



Like enviado a Erling Haaland



Like enviado a Sergio Ruiz Obama



*SELECT * FROM matches antes del match*

Cuando el frontend envía un like, el backend lo inserta en la tabla likes. El trigger de la base de datos detecta automáticamente si hay reciprocidad e inserta el match. El backend entonces consulta la tabla matches para comprobar si se ha generado un match y, en caso afirmativo, genera el XML y lo incluye en la respuesta JSON.

Generación del XML

```

1  <?xml version="1.0" encoding="UTF-8"?>
2  <xsl:stylesheet version="1.0"
3      xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
4      <xsl:output method="html" encoding="UTF-8" indent="yes"/>
5      <xsl:template match="/match">
6          <html lang="es">
7              <head>
8                  <meta charset="UTF-8"/>
9                  <title>LoveCode - Ficha de Match</title>
10                 <style>
11                     body {
12                         font-family: Arial, sans-serif;
13                         background: #f9f9f9;
14                         display: flex;
15                         justify-content: center;
16                         padding: 2rem;
17                         margin: 0;
18                     }
19                     .ficha {
20                         background: #fff;
21                         border: 1px solid #ddd;
22                         border-radius: 12px;
23                         padding: 2rem;
24                         max-width: 600px;
25                         width: 100%;
26                         box-shadow: 0 4px 12px rgba(0,0,0,.08);
27                     }
28                     h1 { color: #e05c5c; text-align: center; margin-bottom: .4rem; }
29                     .fecha { text-align: center; font-size: 12px; color: #aaa; margin-bottom: 2rem; }
30                     .usuarios { display: flex; gap: 1.5rem; margin-bottom: 2rem; }
31                     .usuario {

```

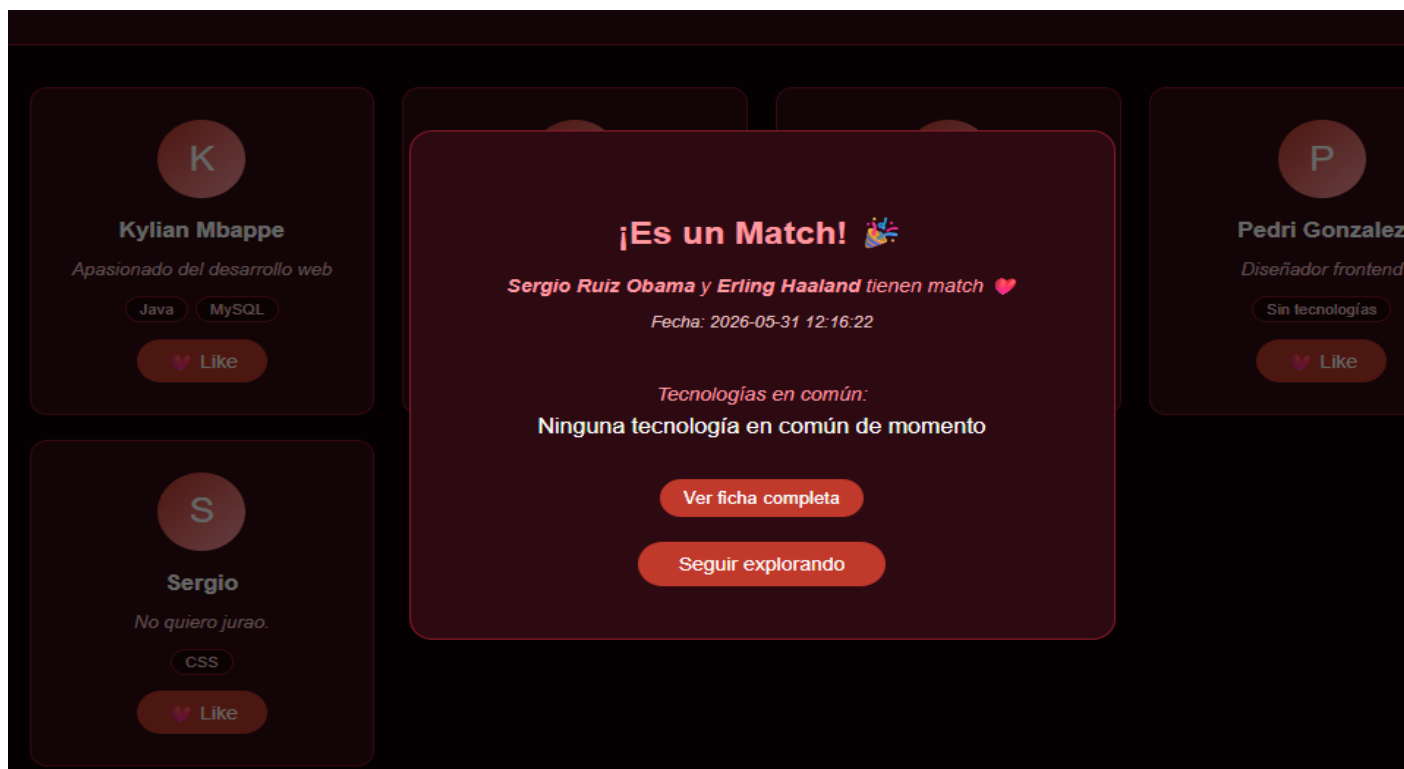
Archivo match.xsl – estructura XSLT (parte 1)

```

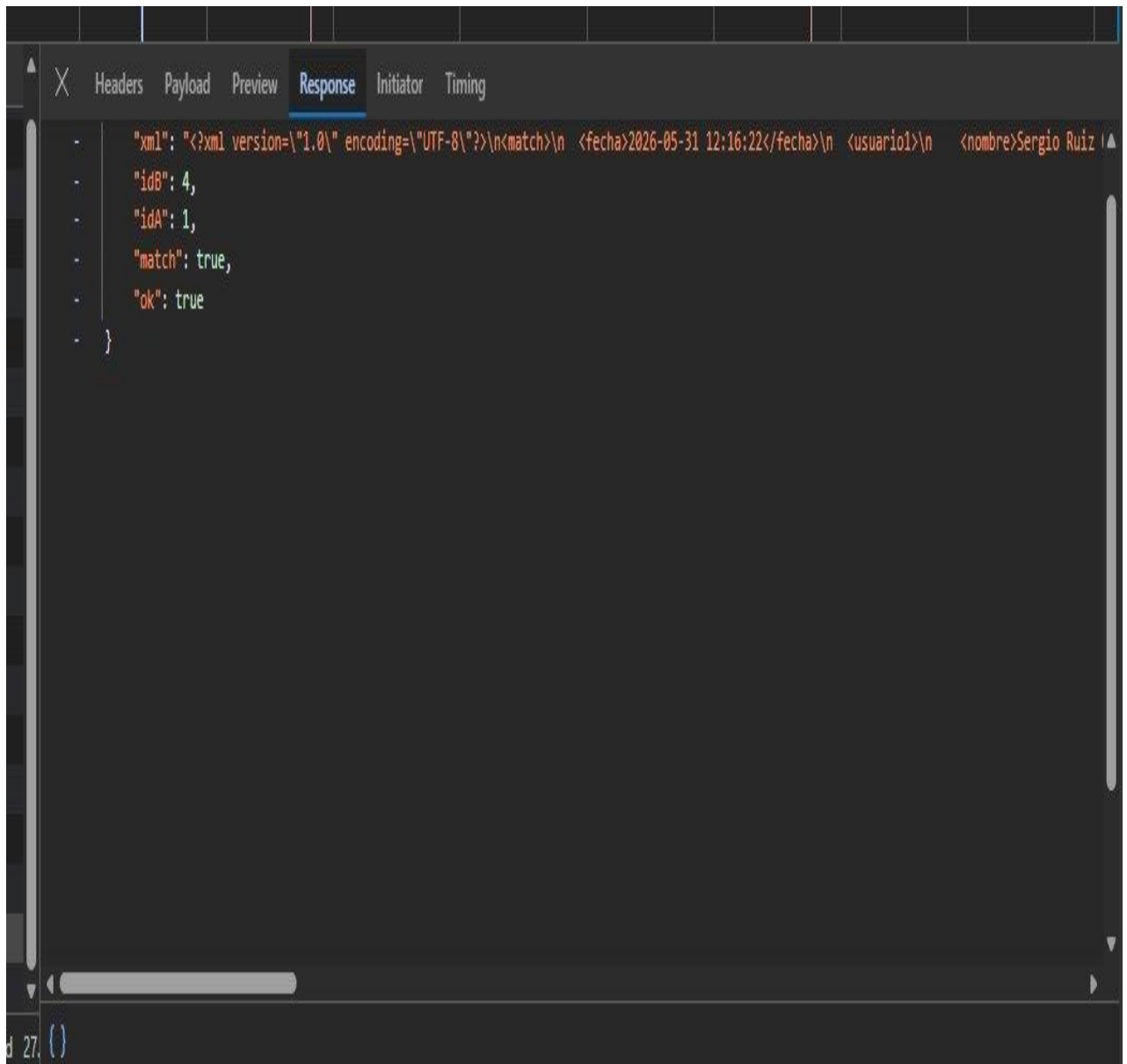
.usuarios { display: flex; gap: 1.5rem; margin-bottom: 2rem; }
.usuario {
  flex: 1;
  background: #f9f9f9;
  border: 1px solid #eee;
  border-radius: 8px;
  padding: 1rem;
  text-align: center;
}
.usuario h3 { color: #e05c5c; margin-bottom: .5rem; font-size: 1rem; }
.usuario p { font-size: 12px; color: #777; margin-bottom: .8rem; }
.tags { display: flex; flex-wrap: wrap; gap: 5px; justify-content: center; }
.tag {
  background: #f4f6fb;
  color: #555;
  border: 1px solid #ddd;
  font-size: 11px;
  padding: 3px 9px;
  border-radius: 20px;
}
.tag.verde { background: #eafaf1; color: #1e8449; border-color: #a9dfbf; }
.comunes { text-align: center; }
.comunes h3 { color: #555; margin-bottom: 1rem; font-size: .95rem; }
.volver {
  display: block;
  text-align: center;
  margin-top: 1.5rem;

```

Archivo match.xml – estilos CSS de la ficha (parte 2)



¡Es un Match! generado en el frontend



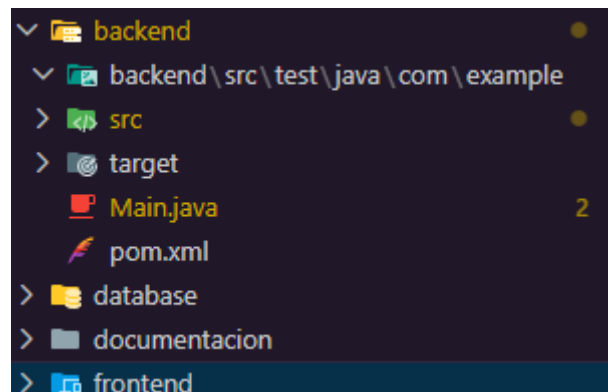
```
Headers Payload Preview Response Initiator Timing
- "xml": "<?xml version='1.0' encoding='UTF-8'?>\n<match>\n  <fecha>2026-05-31 12:16:22</fecha>\n  <usuario1>\n  <nombre>Sergio Ruiz
- "idB": 4,
- "idA": 1,
- "match": true,
- "ok": true
- }
```

Ficha de compatibilidad generada por XSLT

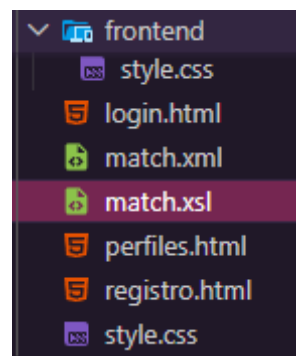
Cuando se produce un match, el backend genera un archivo XML con la información de ambos usuarios: nombre, descripción, tecnologías y fecha del match. También calcula las tecnologías en común mediante un JOIN entre las tablas usuarios_tecnologías de ambos usuarios. Este XML es procesado por el frontend mediante una transformación XSLT para mostrar la ficha de compatibilidad.

5. Desarrollo del Frontend

Estructura general



Estructura del backend Java en VSCode



Archivos del frontend y scripts

El frontend está formado por cuatro páginas HTML: login.html, registro.html, perfiles.html y una ficha de compatibilidad generada desde match.xml y match.xsl. Todas comparten el mismo archivo de estilos style.css, que define el diseño visual de la aplicación con una paleta de colores oscura en tonos rojos.

Formulario de registro

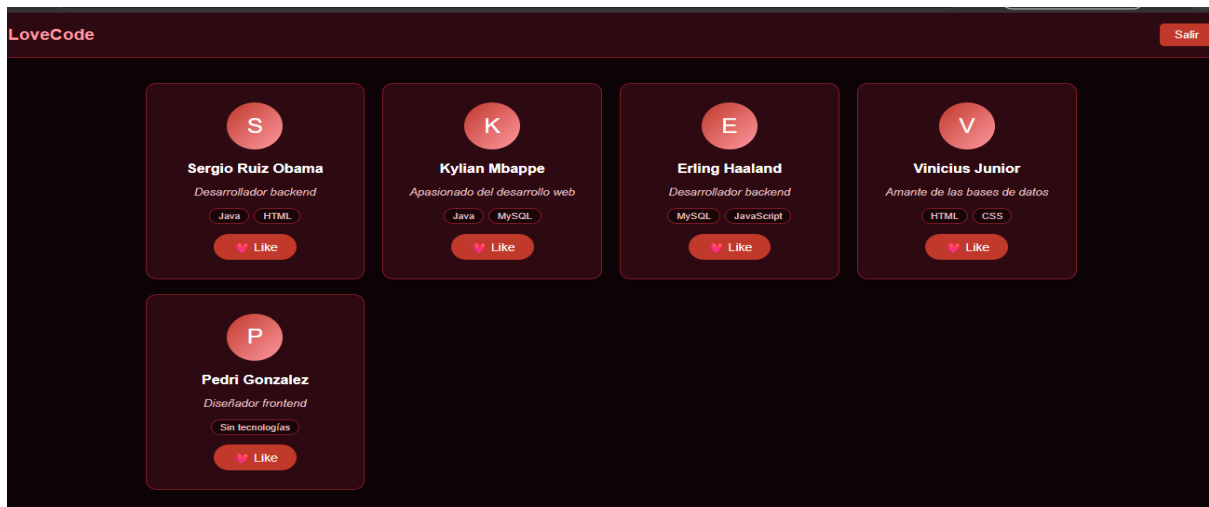
La página de registro permite al usuario introducir su nombre, email, contraseña y descripción. Las tecnologías disponibles se cargan dinámicamente desde el servidor mediante una petición GET a /tecnologías, y se muestran como checkboxes que el usuario puede seleccionar. Al enviar el formulario, se realiza una petición POST a /registro con los datos en formato JSON.

Formulario de login

La página de login solicita el email y la contraseña del usuario. Al enviar el formulario, se realiza una petición POST a /login. Si las credenciales son correctas,

el servidor devuelve el id del usuario, que se guarda en sessionStorage para mantener la sesión activa durante la navegación.

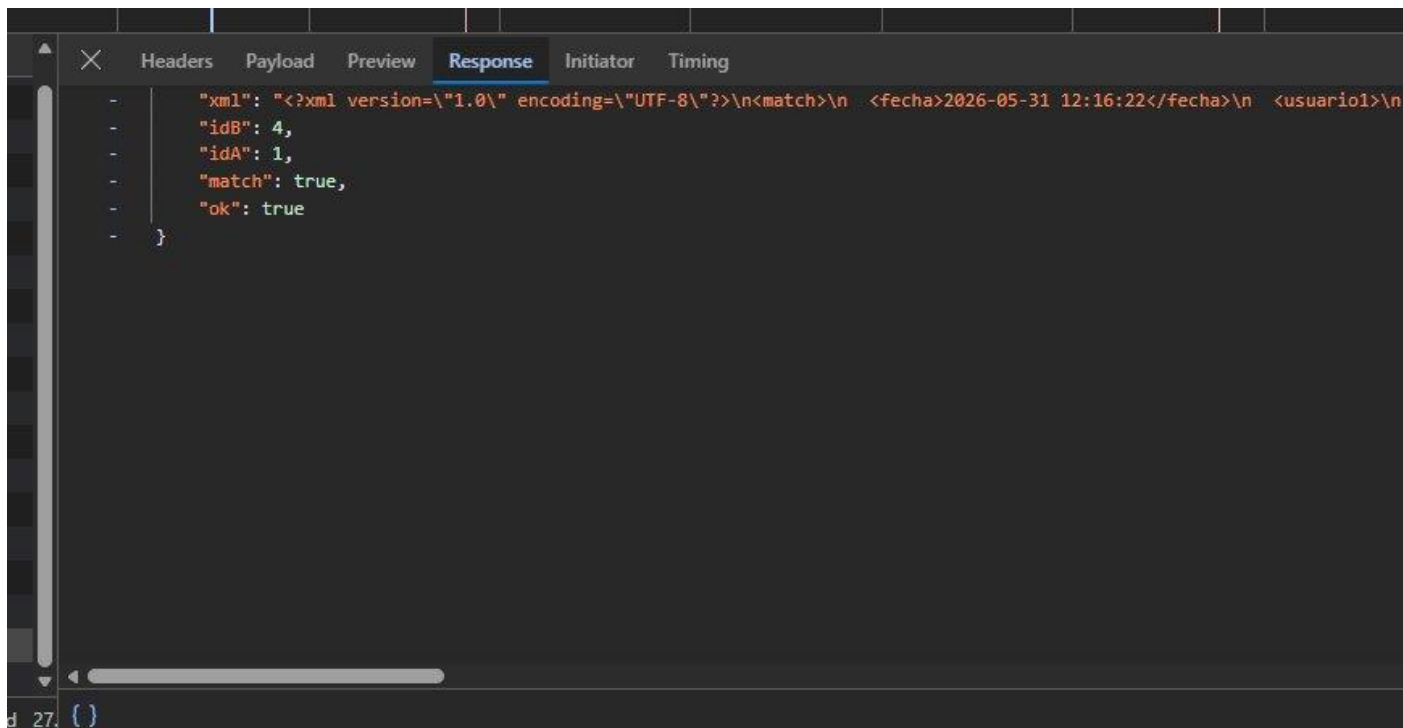
Vista de perfiles



Pantalla de perfiles con tarjetas de usuarios

La página de perfiles muestra las tarjetas de los demás usuarios generadas dinámicamente mediante JavaScript. Cada tarjeta incluye el nombre, la descripción y las tecnologías del perfil, así como un botón de like. Al cargar la página se realiza una petición GET a /perfiles con el id de sesión para obtener los perfiles disponibles.

Sistema de likes y matches



DevTools Network: POST /like con respuesta match:true

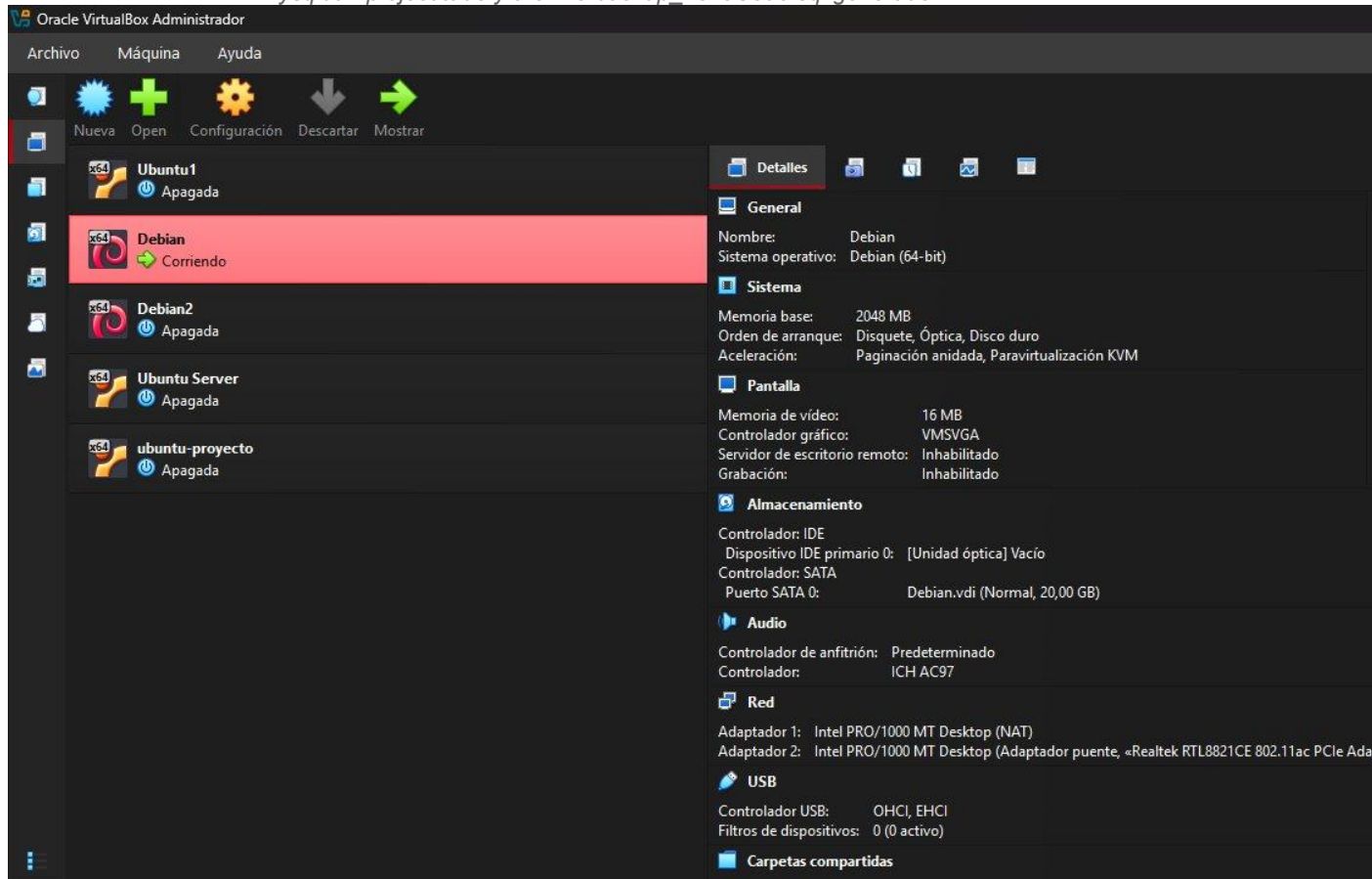
Al pulsar el botón de like, se realiza una petición POST a /like con el id del emisor y el receptor. Si la respuesta indica que se ha producido un match, se muestra un modal con la información del match y un botón para ver la ficha de compatibilidad completa. La ficha se abre en una nueva pestaña y muestra el XML transformado mediante XSLT.

6. Automatización del Sistema

Script de backup

```
oscarrrl@debian:~$ sudo mysqldump -u root -p LoveCode > backup_LoveCode.sql
[sudo] contraseña para oscarrrl:
Enter password:
oscarrrl@debian:~$ ls -lh ~/backup_LoveCode.sql
-rw-r--r-- 1 oscarrrl oscarrrl 1.2K Feb 10 12:12 /home/oscarrrl/backup_LoveCode.sql
oscarrrl@debian:~$
```

mysqldump ejecutado y archivo backup_LoveCode.sql generado



VM Debian corriendo en VirtualBox con MariaDB activo

```

oscarrrl@debian:~$ sudo systemctl status mariadb
[sudo] contraseña para oscarrrl:
● mariadb.service - MariaDB 11.8.6 database server
   Loaded: loaded (/usr/lib/systemd/system/mariadb.service; enabled; pre
   Active: active (running) since Sat 2026-05-30 18:03:53 CEST; 23h ago
   Invocation: cf21b11abd114368bac4d30f3ee2a8a0
     Docs: man:mariadb(8)
           https://mariadb.com/kb/en/library/systemd/
   Process: 843 ExecStartPre=/bin/sh -c [ ! -e /usr/bin/galera_recovery ]
   Process: 1019 ExecStartPost=/bin/rm -f /run/mysqld/wsrep-start-positio
   Process: 1021 ExecStartPost=/etc/mysql/debian-start (code=exited, stat
 Main PID: 911 (mariabdb)
   Status: "Taking your SQL requests now..."
     Tasks: 19 (limit: 15165)
    Memory: 173M (peak: 173.4M)
       CPU: 42.885s
    CGroup: /system.slice/mariadb.service
           └─911 /usr/sbin/mariabdb

```

MariaDB activo en la VM Debian (systemctl status mariadb)

7. Funcionamiento del Proyecto

El flujo completo de uso de la aplicación es el siguiente:

Registro de usuario

El usuario accede a la página de registro, rellena sus datos y selecciona las tecnologías que domina. Al pulsar Crear cuenta, los datos se envían al servidor, que los almacena en la base de datos y redirige al usuario a la página de perfiles.

Inicio de sesión

El usuario introduce su email y contraseña en la página de login. El servidor valida las credenciales y, si son correctas, redirige al usuario a la página de perfiles con su sesión iniciada.

Visualización de perfiles

La página de perfiles muestra las tarjetas de todos los usuarios registrados excepto el propio. Cada tarjeta muestra el nombre, la descripción y las tecnologías del perfil.

Dar like y generar un match

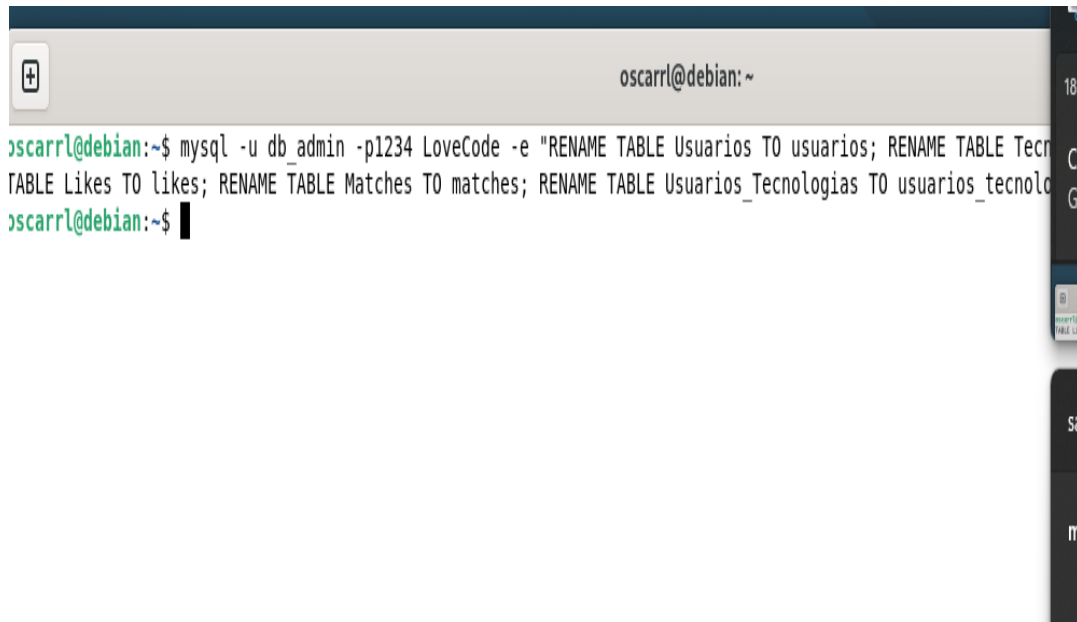
Al pulsar el botón Like en una tarjeta, se registra el like en la base de datos. Si el otro usuario también ha dado like, el trigger genera automáticamente un match y el sistema muestra un modal de celebración con la información del match y las tecnologías en común.

Ficha de compatibilidad

Al pulsar Ver ficha completa en el modal de match, se abre una nueva pestaña que muestra la ficha de compatibilidad generada a partir del XML del match transformado mediante XSLT. La ficha muestra los datos de ambos usuarios y sus tecnologías.

8. Problemas Encontrados y Soluciones

Diferencia de mayúsculas en nombres de tablas: MariaDB en Linux distingue entre mayúsculas y minúsculas en los nombres de tablas. Las tablas estaban creadas con mayúscula inicial (Usuarios, Tecnologías...) pero el código las buscaba en minúscula. La solución fue renombrar todas las tablas a minúsculas con el comando RENAME TABLE.

A terminal window titled 'oscarrrl@debian: ~' showing a series of MySQL commands. The user runs 'mysql -u db_admin -p1234 LoveCode -e "RENAME TABLE Usuarios TO usuarios; RENAME TABLE Tecnologías TO tecnologías; RENAME TABLE Likes TO likes; RENAME TABLE Matches TO matches; RENAME TABLE Usuarios_Tecnologías TO usuarios_tecnologías;"'. The prompt returns to the shell.

```
oscarrrl@debian: ~  
oscarrrl@debian:~$ mysql -u db_admin -p1234 LoveCode -e "RENAME TABLE Usuarios TO usuarios; RENAME TABLE Tecnologías TO tecnologías; RENAME TABLE Likes TO likes; RENAME TABLE Matches TO matches; RENAME TABLE Usuarios_Tecnologías TO usuarios_tecnologías;"  
oscarrrl@debian:~$
```

Tablas renombradas a minúsculas correctamente

Columnas con nombres distintos: las columnas `id_usuario` e `id_tecnologia` no coincidían con los nombres que esperaba el código. Se resolvió renombrando las columnas a `id` mediante ALTER TABLE.

Trigger con referencias en mayúscula: el trigger original usaba Likes y Matches con mayúscula. Al renombrar las tablas el trigger dejó de funcionar. Se recreó el trigger desde la terminal de MySQL con los nombres correctos en minúscula.

Port forwarding en VirtualBox: la base de datos corre en una VM y el backend en Windows. Se configuró correctamente el reenvío del puerto 3307 de Windows al 3306 de la VM para que la conexión JDBC funcionara.

```
-----  
scarrl@debian:~$ hostname-I  
ash: hostname-I: orden no encontrada  
scarrl@debian:~$ ip a | grep inet  
    inet 127.0.0.1/8 scope host lo  
    inet6 ::1/128 scope host noprefixroute  
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute enp0s3  
    inet6 fd17:625c:f037:2:11fd:1253:a064:c0fa/64 scope global temporary dynan  
  
    inet6 fd17:625c:f037:2:a00:27ff:fe60:8400/64 scope global dynamic mngtmpac  
noprefixroute  
    inet6 fe80::a00:27ff:fe60:8400/64 scope link noprefixroute  
scarrl@debian:~$ █
```

IP de la VM Debian obtenida con ip a | grep inet

Conexión JDBC con las tablas renombradas: tras renombrar las columnas, fue necesario actualizar las queries SQL en el código Java para que usaran los nuevos nombres de columnas.

9. Conclusiones

El desarrollo del proyecto LoveCode me ha permitido poner en práctica de forma integrada los conocimientos adquiridos en las distintas asignaturas del ciclo formativo, comprendiendo cómo se relacionan entre sí la base de datos, el backend y el frontend en una aplicación web real.

Las partes más complejas de la resolución de los problemas de compatibilidad entre el código Java y la base de datos MariaDB en Linux, especialmente por las diferencias en el tratamiento de mayúsculas y minúsculas.

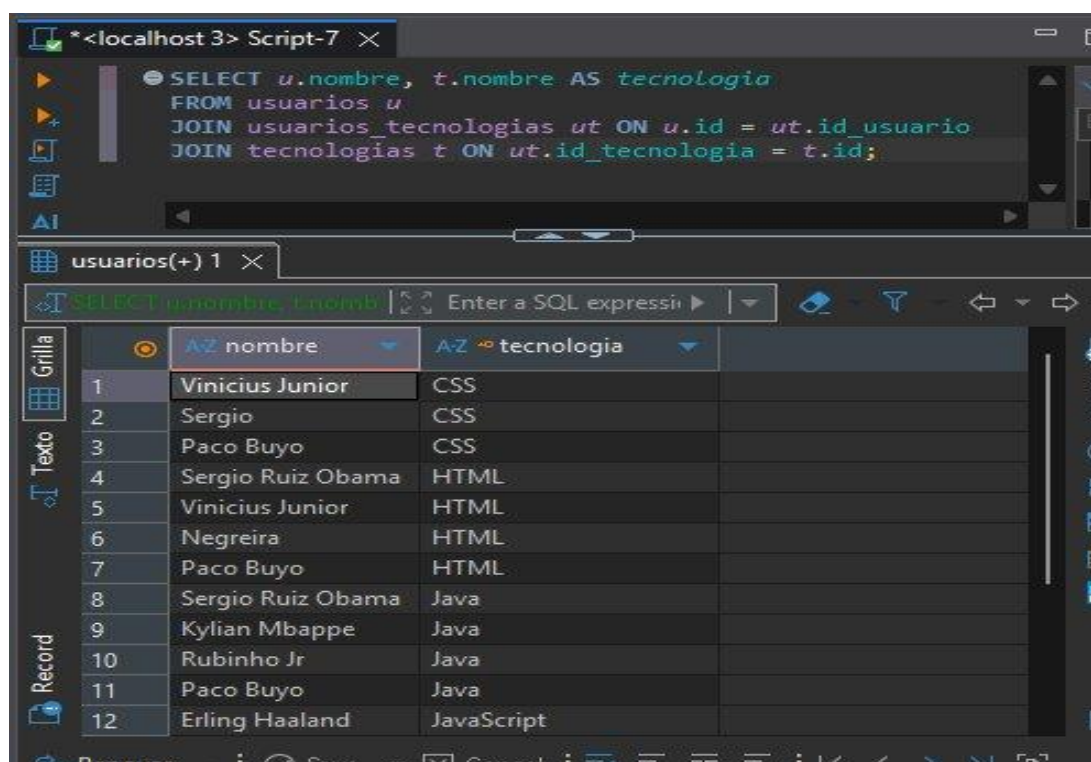
10. Anexos

10.1 Consultas SQL avanzadas

A continuación se muestran ocho consultas ejecutadas sobre la base de datos del proyecto, cada una de un tipo distinto.

Consulta 1 — JOIN: cada usuario con su tecnología

Se combinan tres tablas para ver que tecnología tiene registrada cada usuario. El resultado muestra el nombre del usuario y el nombre de su tecnología.



The screenshot shows a SQL IDE window titled '*<localhost 3> Script-7'. The SQL editor contains the following query:

```
SELECT u.nombre, t.nombre AS tecnologia
FROM usuarios u
JOIN usuarios_tecnologias ut ON u.id = ut.id_usuario
JOIN tecnologías t ON ut.id_tecnologia = t.id;
```

Below the editor, the results are displayed in a table view. The table has two columns: 'nombre' and 'tecnologia'. The results are as follows:

	nombre	tecnologia
1	Vinicius Junior	CSS
2	Sergio	CSS
3	Paco Buyo	CSS
4	Sergio Ruiz Obama	HTML
5	Vinicius Junior	HTML
6	Negreira	HTML
7	Paco Buyo	HTML
8	Sergio Ruiz Obama	Java
9	Kylian Mbappe	Java
10	Rubinho Jr	Java
11	Paco Buyo	Java
12	Erling Haaland	JavaScript

Figura. Cada usuario aparece junto a la tecnología que tiene asociada en la base de datos.

Consulta 2 — COUNT + GROUP BY: número de tecnologías por usuario

Se cuenta cuántas tecnologías tiene cada usuario. El resultado agrupa los datos por nombre de usuario y muestra el total de cada uno.

Script-7

```
SELECT u.nombre, COUNT(*) AS total_tecnologias
FROM usuarios u
JOIN usuarios_tecnologias ut ON u.id = ut.id_usuario
GROUP BY u.nombre;
```

usuarios 1

Grilla

	A-Z nombre	123 total_tecnologias
1	Erling Haaland	2
2	Kylian Mbappe	2
3	Negreira	2
4	Paco Buyo	5
5	Rubinho Jr	2
6	Sergio	1
7	Sergio Ruiz Obama	2
8	Vinicius Junior	2

Record

Figura. Número de tecnologías que tiene cada usuario registrado, ordenado alfabéticamente.

Consulta 3 — ORDER BY: lista de usuarios ordenada alfabéticamente

Se listan el nombre y el email de todos los usuarios, ordenados de la A a la Z por nombre.

Script-7

```
SELECT nombre, email FROM usuarios ORDER BY nombre ASC;
```

usuarios 1

Grilla

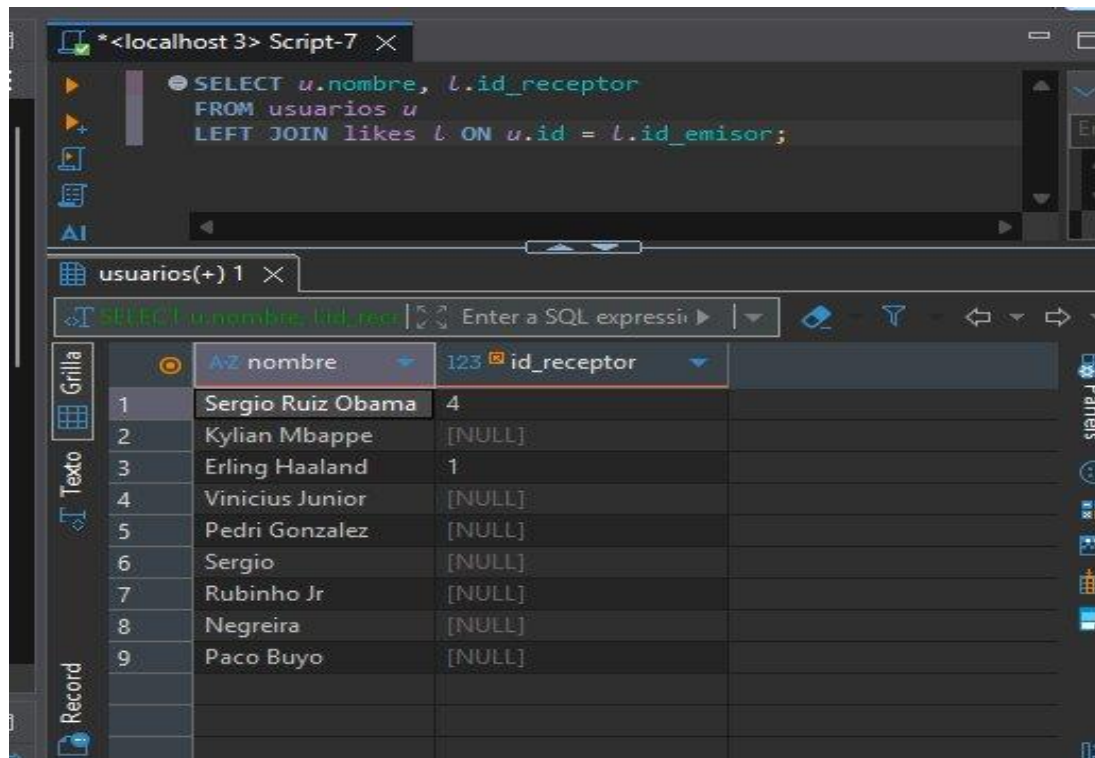
	A-Z nombre	A-Z email
1	Erling Haaland	haaland@email.com
2	Kylian Mbappe	mbappe@email.com
3	Negreira	nosoycorrupto@gmail.com
4	Paco Buyo	paquito@gmail.com
5	Pedri Gonzalez	pedri@email.com
6	Rubinho Jr	carls@gmail.com
7	Sergio	sergio@gmailuknow.com
8	Sergio Ruiz Obama	sergio.ruiz.obama@gmail.com
9	Vinicius Junior	vinicius@email.com

Record

Figura. Lista de usuarios con su email, ordenados alfabéticamente por nombre.

Consulta 4 — LEFT JOIN: usuarios y los likes que han enviado

Se usa un LEFT JOIN para mostrar todos los usuarios junto con el identificador de la persona a la que han dado like. Los que no han dado ningún like aparecen con el valor NULL.



```

SELECT u.nombre, l.id_receptor
FROM usuarios u
LEFT JOIN likes l ON u.id = l.id_emisor;

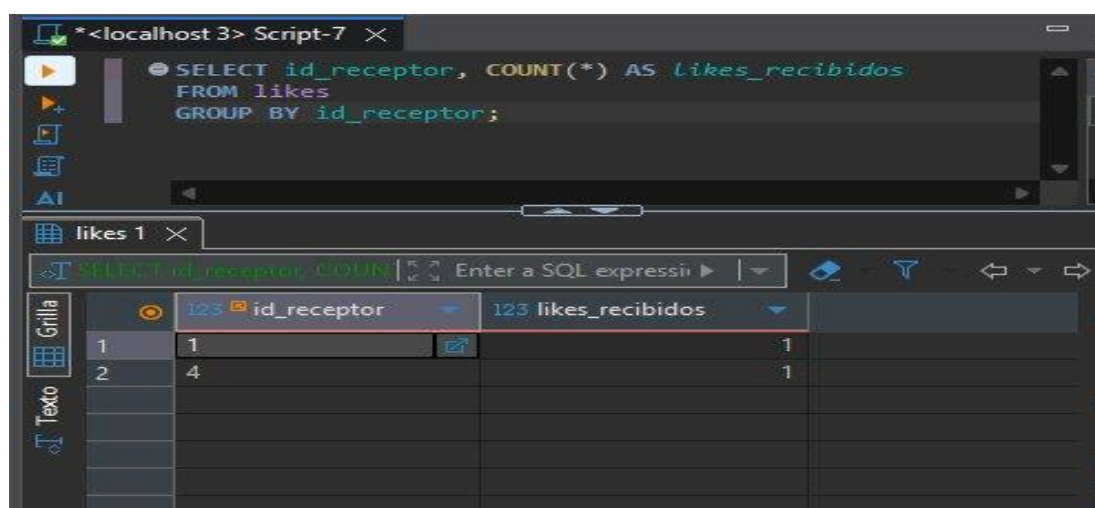
```

	A-Z nombre	123 id_receptor
1	Sergio Ruiz Obama	4
2	Kylian Mbappe	[NULL]
3	Erling Haaland	1
4	Vinicius Junior	[NULL]
5	Pedri Gonzalez	[NULL]
6	Sergio	[NULL]
7	Rubinho Jr	[NULL]
8	Negreira	[NULL]
9	Paco Buyo	[NULL]

Figura. Todos los usuarios y a quien han dado like. Los que no han dado ninguno aparecen con NULL.

Consulta 5 — COUNT: likes recibidos por cada usuario

Se cuentan los likes que ha recibido cada usuario, agrupando la tabla de likes por receptor. Solo aparecen los usuarios que han recibido algún like.



```

SELECT id_receptor, COUNT(*) AS Likes_recibidos
FROM likes
GROUP BY id_receptor;

```

	123 id_receptor	123 likes_recibidos
1	1	1
2	4	1

Figura. Número de likes recibidos por cada usuario (solo aparecen los que han recibido alguno).

Consulta 6 — VISTA: creación y consulta de una vista

Se crea una vista llamada vista_usuarios_tecnologias. Una vista es como guardar una consulta para poder usarla después sin repetirla. La pantalla muestra que la creación se completó correctamente en 0,083 segundos.

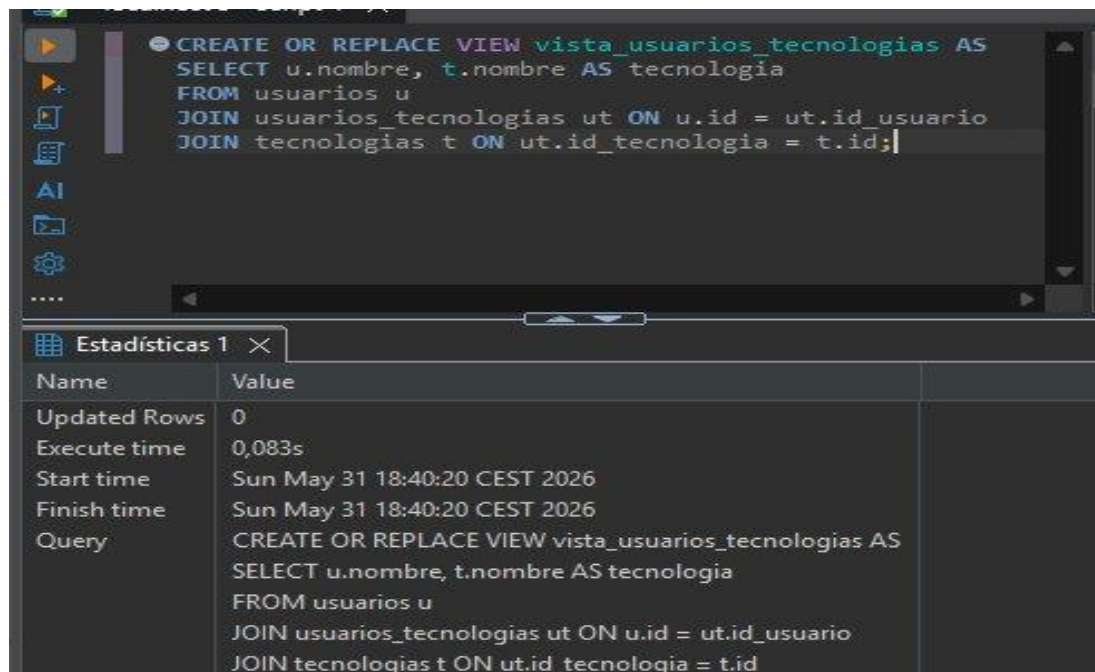


Figura. Confirmación de que la vista se creó correctamente en la base de datos.

A continuación se consulta la vista recién creada para comprobar que devuelve los datos correctamente.

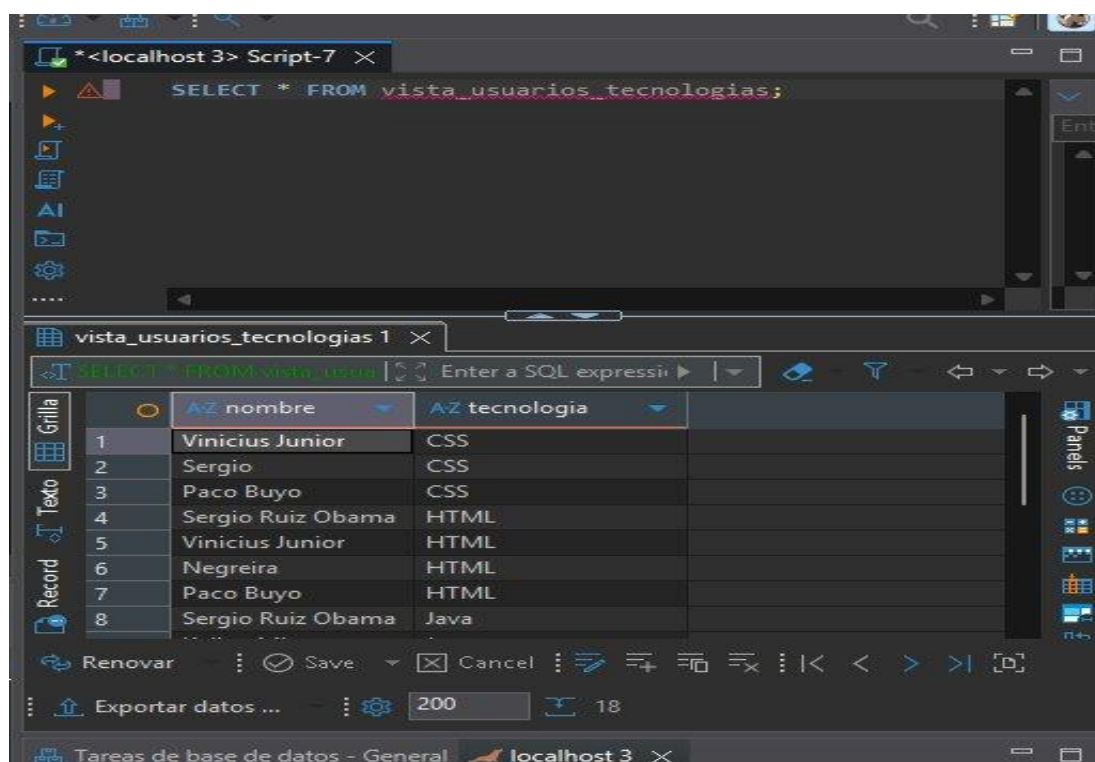


Figura. Resultado de consultar la vista: cada usuario aparece con su tecnología correspondiente.

Consulta 7 — INDICE: creación de un índice en la tabla likes

Se crea un índice llamado `idx_likes_emisor` sobre la columna `id_emisor` de la tabla `likes`. Después se ejecuta `SHOW INDEX FROM likes;` para comprobar que el índice se creó correctamente junto con los demás índices de la tabla.

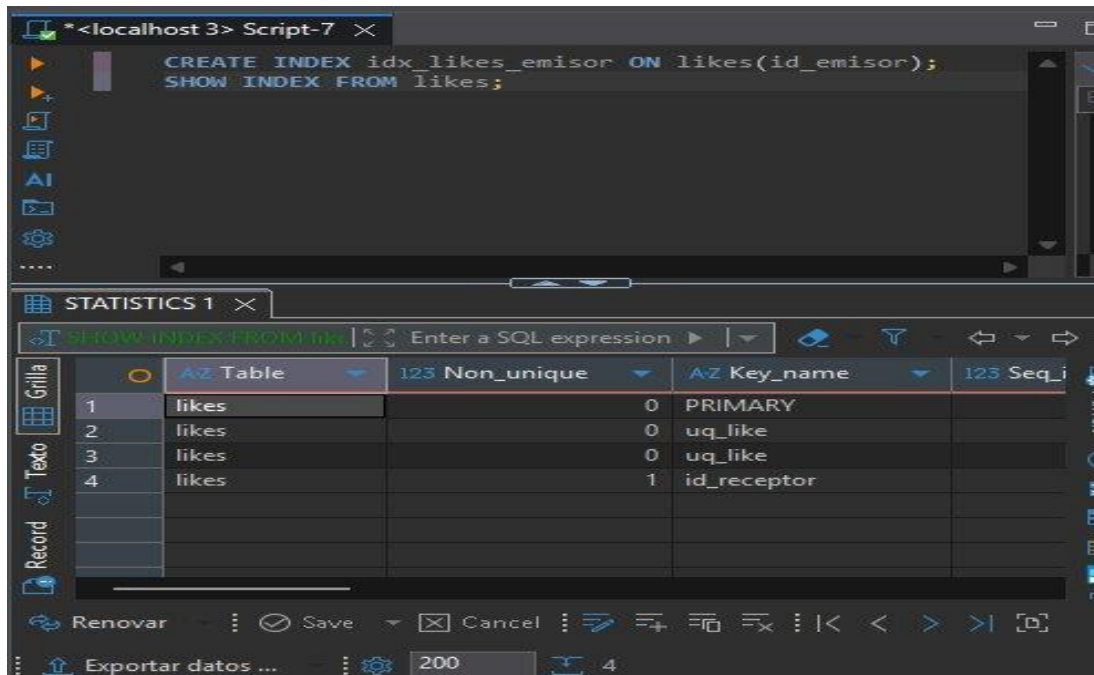


Figura. Lista de índices de la tabla `likes`, donde se puede ver el nuevo índice `idx_likes_emisor` creado.

Consulta 8 — Búsqueda usando el índice

Se buscan los registros de `likes` donde el emisor es el usuario con `id` 1. Gracias al índice creado antes, la base de datos encuentra el resultado directamente sin revisar toda la tabla. La consulta devuelve un solo registro.

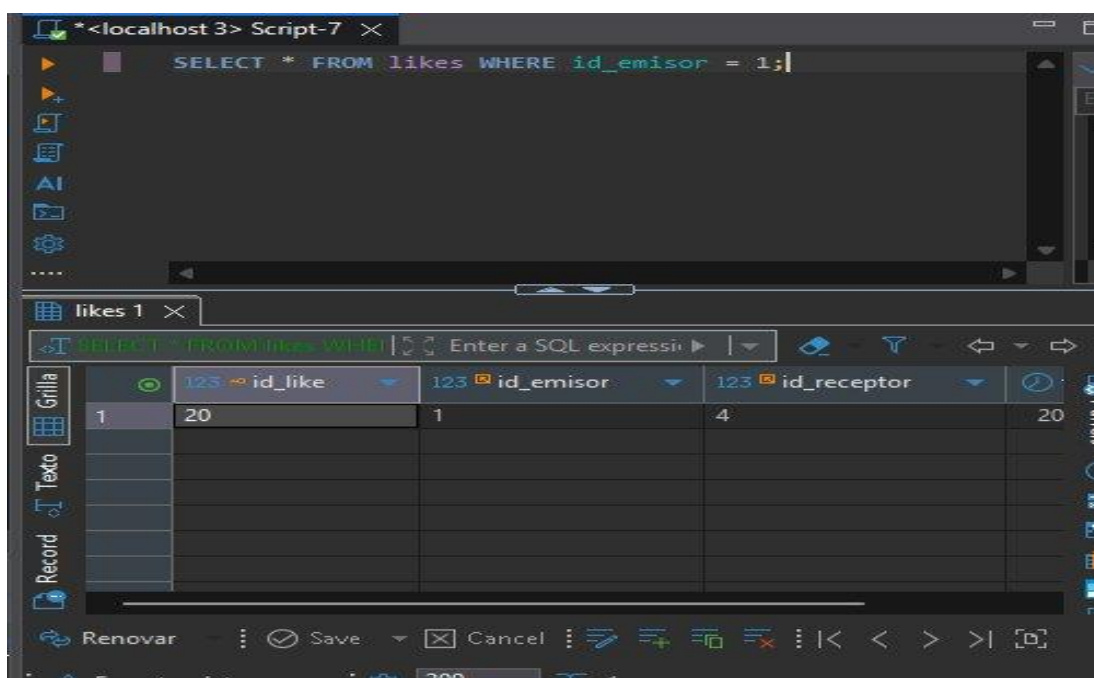


Figura. Resultado de buscar los likes enviados por el usuario 1. El índice hace que la búsqueda sea más rápida.

10.2 Tests unitarios

Se han creado seis pruebas automáticas en Java con JUnit para comprobar que el registro, el login y los likes funcionan como se espera. En la captura se ve que las seis pruebas pasan correctamente sin ningún error.

```
[INFO] -----  
[INFO] T E S T S  
[INFO] -----  
[INFO] Running com.example.LoveCodeTest  
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.302 s -- in com.example.LoveCodeTest  
[INFO]  
[INFO] Results:  
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0  
[INFO]  
[INFO] -----  
[INFO] BUILD SUCCESS  
[INFO] -----  
[INFO] Total time: 26.226 s  
[INFO] Finished at: 2026-05-31T19:28:06+02:00  
[INFO] -----
```

Figura. Resultado de ejecutar los tests con Maven: 6 pruebas correctas, 0 errores, BUILD SUCCESS.

10.3 Repositorio de GitHub

El proyecto está subido en un repositorio público de GitHub donde se puede ver el código organizado por carpetas y el historial de todos los cambios realizados.

Estructura del repositorio

El repositorio tiene tres carpetas principales: backend con el código Java, frontend con las páginas HTML y database con el SQL de la base de datos.

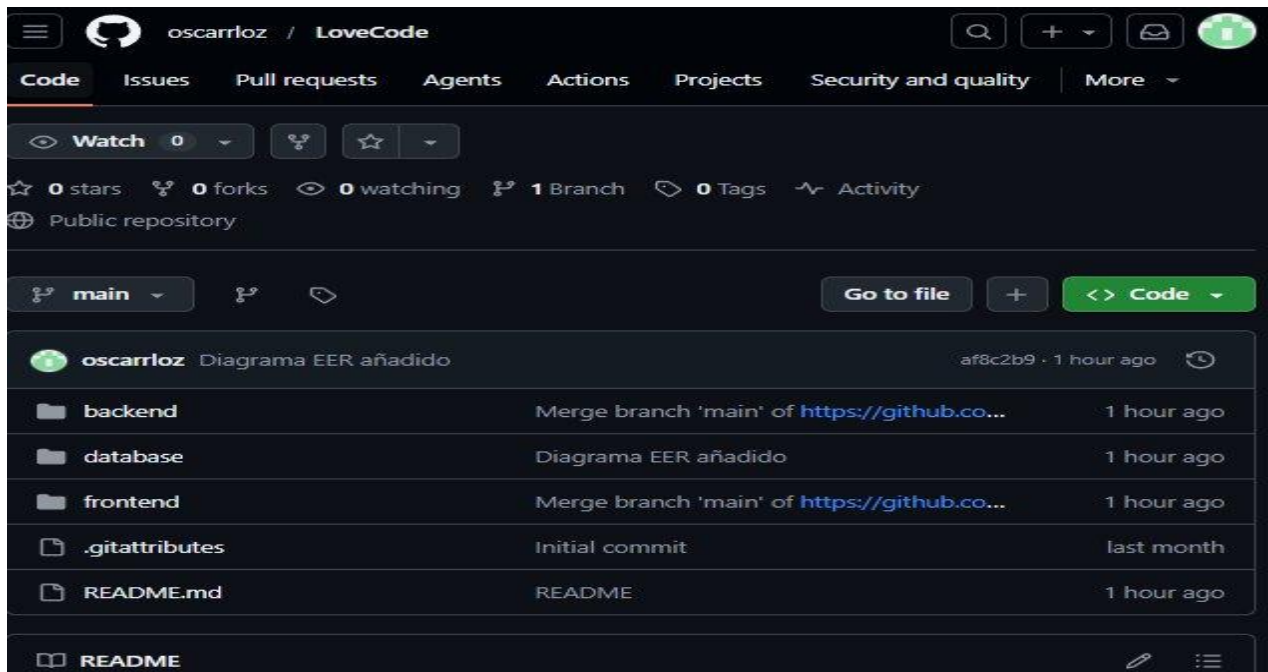


Figura. Vista principal del repositorio en GitHub con las carpetas del proyecto.

Historial de cambios (commits)

Cada vez que se hizo un cambio en el proyecto se guardó con un mensaje describiendo que se hizo. Aquí se ve el historial completo de cambios.

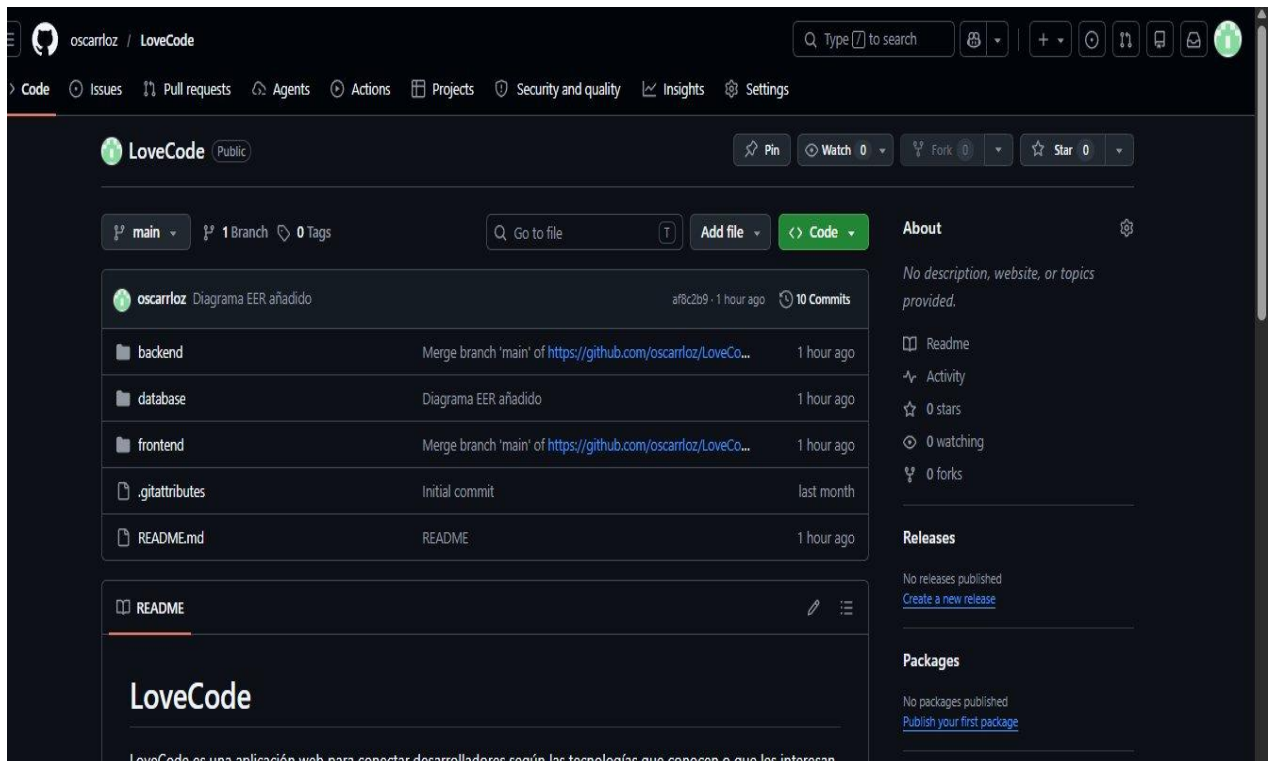


Figura. Historial de commits del repositorio con los mensajes de cada cambio.

README del proyecto

La página principal del repositorio muestra el archivo README con la descripción del proyecto, las tecnologías usadas y las instrucciones para ejecutarlo.



Figura. README del proyecto tal como se ve en la página principal del repositorio en GitHub.

Contribuciones al repositorio

La sección de Contributors de GitHub muestra una gráfica con las aportaciones realizadas al repositorio y el número total de commits.

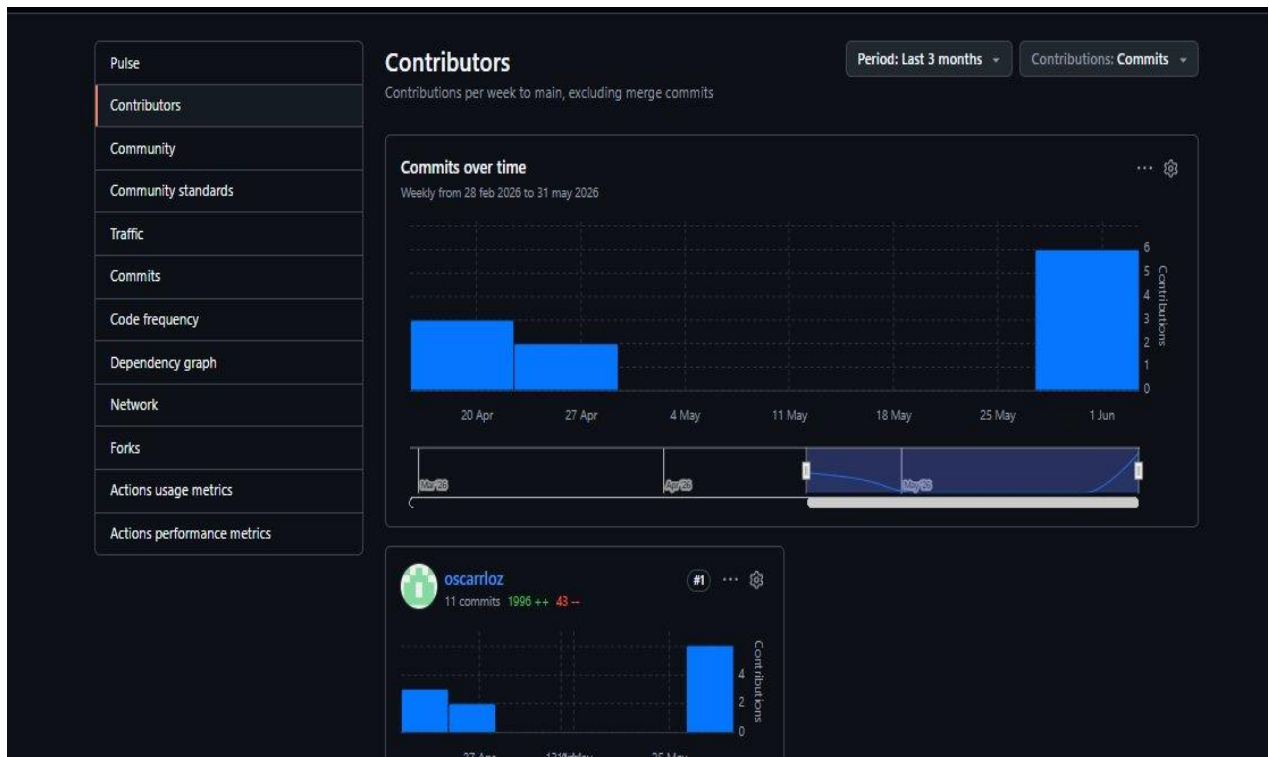


Figura. Pantalla de contribuciones al repositorio, con la gráfica de actividad y el total de commits.